

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO5: Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems.(L4,L5)

Assessment Tool : Alternative Solution Design

Weightage: 20%

QUESTIONS:

Total Marks: 25

- A company needs to develop a web service for a mobile and web application. Analyze both **REST and SOAP** approaches and design an appropriate solution by selecting one. Justify your choice based on performance, scalability, and security requirements.
- An online platform expects a large number of users accessing its services simultaneously. Design a **scalable service architecture** showing how client, server, API, and database interact. Propose an alternative design to improve scalability and justify your approach.
- A banking application requires secure communication between client and server. Analyze available communication protocols and **select the most suitable protocol**. Justify your selection based on security, reliability, and performance.
- An application needs to integrate with third-party services such as payment gateways or maps. Design an **API integration strategy**, including request handling, authentication, and response processing. Suggest an alternative approach to improve efficiency or reliability.
- A web service frequently encounters errors during client requests. Design an **error handling and fault tolerance mechanism**. Propose alternative strategies to improve system reliability and ensure smooth user experience.

Code of Conduct:

I Ebin Antony P (192373011) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this

assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weight age (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly defines problem, objectives, and all requirements accurately	Understands problem with minor gaps	Partial understanding; misses key requirements	Misinterprets or fails to understand problem
Generation of Solutions	25%	Develops multiple innovative and feasible alternatives with clear differentiation	Generates relevant alternatives with minor limitations	Limited or repetitive alternatives	Fails to generate meaningful alternatives
Evaluation of Alternatives	25%	Critically compares alternatives using appropriate criteria and metrics	Compares alternatives with some justification	Basic or superficial comparison	No proper comparison conducted
Solution Selection	20%	Selects best solution with strong justification and decision rationale	Selects suitable solution with moderate justification	Selection is weakly justified	No clear or justified selection

Alternative Solution Design – Web Services

1. REST vs SOAP for Mobile and Web Application

A company needs a web service that can be accessed by both mobile and web applications. The two major approaches are **REST** and **SOAP**.

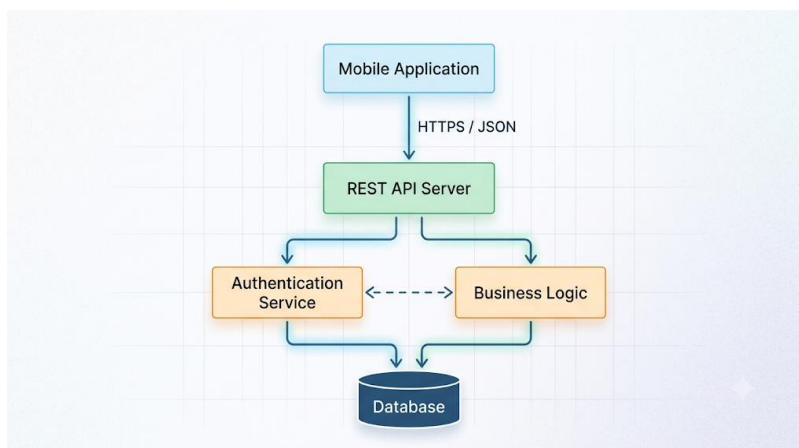
REST vs SOAP

Feature	REST	SOAP
Architecture	RESTful architectural style	XML-based messaging protocol
Data format	JSON, XML, etc.	Mainly XML
Performance	Faster and lightweight	Relatively slower
Scalability	Highly scalable	More complex to scale
Mobile applications	Very suitable	Less suitable
Security	HTTPS, OAuth, JWT	HTTPS, WS-Security
Bandwidth	Low	Higher
Complexity	Simple	More complex
Best suited for	Mobile/web APIs	Enterprise transactions

Proposed Solution: REST API

For a mobile and web application, **REST is the recommended solution** because it is lightweight and provides better performance.

Architecture



Example Request

```
GET /api/users/101
Authorization: Bearer JWT_TOKEN
```

Example Response

```
{
  "id": 101,
  "name": "John",
  "email": "john@example.com"
}
```

Justification

Performance: REST uses lightweight JSON messages and therefore reduces processing and network overhead.

Scalability: REST is stateless, allowing requests to be distributed across multiple servers using load balancing.

Security: HTTPS provides encrypted communication, while OAuth 2.0 or JWT can provide authentication and authorization.

Interoperability: REST APIs can be consumed by Android, iOS, browsers, and different programming languages.

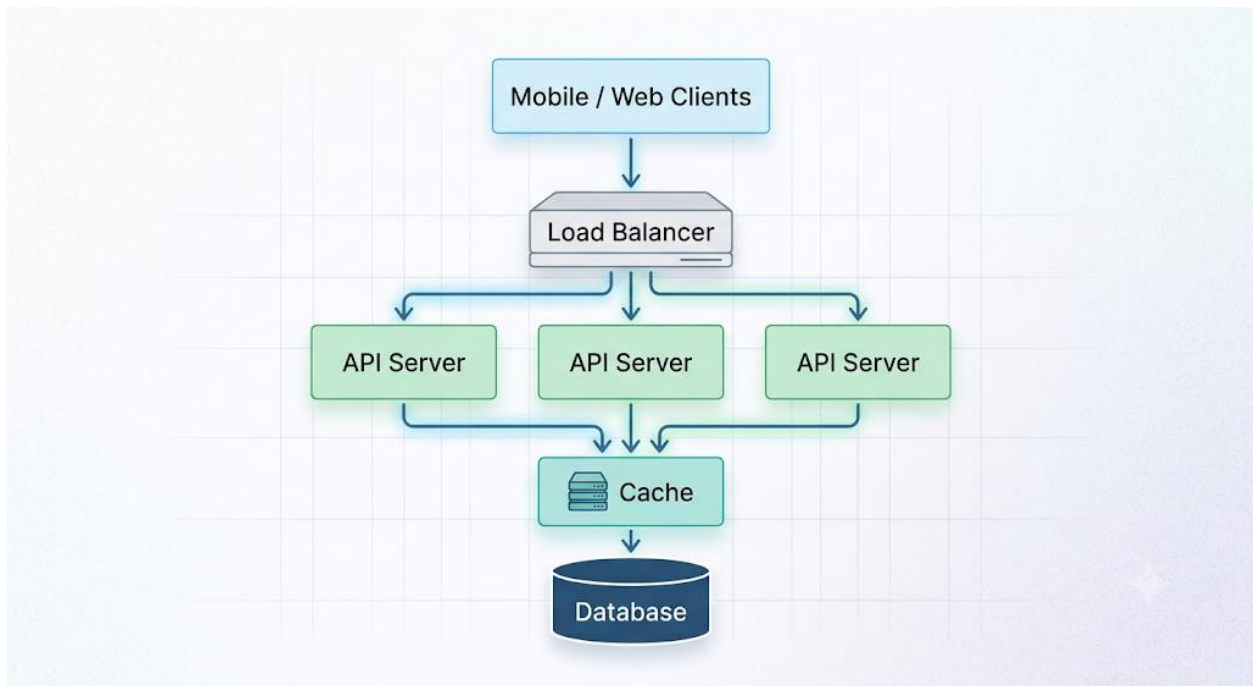
Conclusion

For a mobile and web application, REST provides a better balance of **performance, scalability, simplicity, and security**. SOAP would be preferable when advanced enterprise-level message security or strict WS-* standards are required.

2. Scalable Service Architecture

An online platform with a large number of simultaneous users requires an architecture that can handle high traffic without reducing performance.

Basic Architecture



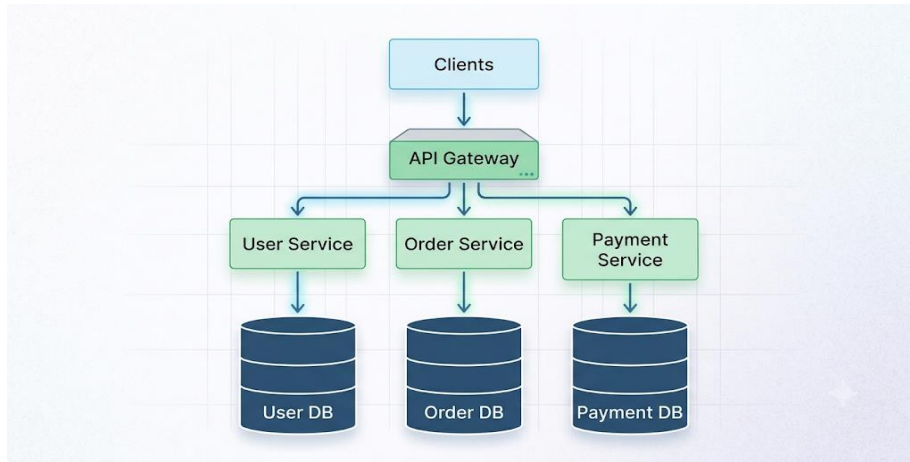
Working

1. Users send requests from mobile or web applications.
2. The load balancer receives incoming traffic.
3. It distributes requests among multiple API servers.
4. API servers process business logic.
5. Frequently requested information is obtained from the cache.
6. Database servers handle persistent data.

7. The response is returned to the client.

Alternative Design: Microservices

A better scalable architecture is to divide the application into independent microservices.



Advantages

- Individual services can scale independently.
- Failure of one service does not necessarily bring down the entire system.
- Services can be deployed independently.
- Load can be distributed efficiently.
- Different technologies can be used for different services.
- Caching and asynchronous message queues can further improve performance.

Conclusion

A combination of **API Gateway + load balancing + multiple service instances + caching + database scaling** provides a highly scalable architecture for large numbers of simultaneous users.

3. Secure Communication for Banking Application

A banking application requires confidentiality, integrity, authentication, and reliable communication.

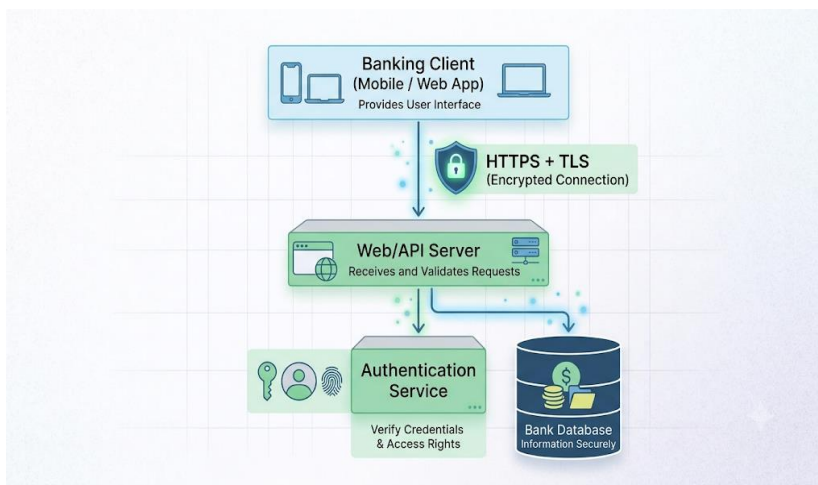
Possible Protocols

Protocol	Security	Performance	Suitability
HTTP	Low	High	Not suitable
HTTPS	High	Good	Highly suitable
FTP	Low/Medium	Good	Not suitable
SMTP	Medium	Good	Not suitable for banking transactions
TLS	Very high	Good	Used to secure communication
WebSocket over TLS	High	Very high	Useful for real-time applications

Selected Protocol: HTTPS with TLS

HTTPS (HTTP Secure) using **TLS (Transport Layer Security)** is the most appropriate choice for banking applications.

Architecture



Security Mechanisms

Encryption: TLS encrypts data while it travels between client and server.

Authentication: Digital certificates allow the client to verify the identity of the server.

Integrity: TLS helps detect modification of transmitted data.

Authentication and Authorization: User authentication can be combined with MFA, OAuth-based mechanisms, or secure session management.

Protection against attacks: HTTPS helps protect against network interception and man-in-the-middle attacks when correctly configured.

Reliability

HTTPS operates over TCP, which provides reliable, ordered delivery of network data.

Performance

Modern TLS versions provide strong security with relatively low overhead. Connection reuse and HTTP/2 or HTTP/3 can further improve performance.

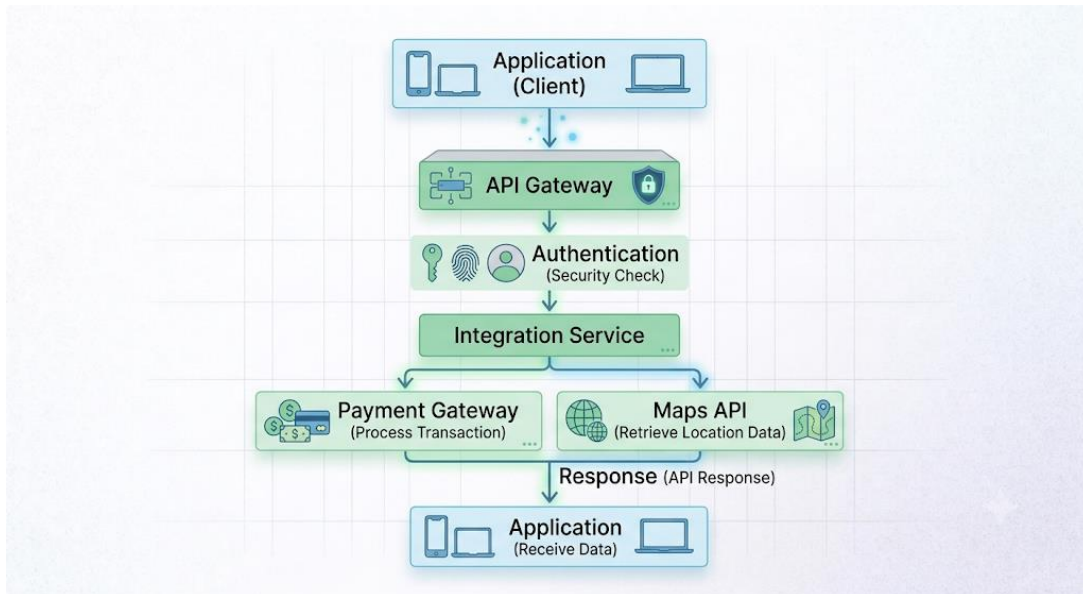
Conclusion

HTTPS secured by modern TLS is the most suitable protocol for banking communication because it provides a strong combination of **security, reliability, and acceptable performance**.

4. Third-Party API Integration Strategy

An application may need to communicate with external services such as payment gateways, map services, SMS providers, or authentication providers.

API Integration Architecture



Request Handling

1. The application receives a user request.
2. The API Gateway validates the request.
3. Authentication credentials are checked.
4. The integration service constructs the third-party API request.
5. The request is sent using HTTPS.
6. The third-party service processes the request.
7. The response is validated.
8. The application converts the response into its required format.
9. The result is returned to the user.

Authentication

Depending on the third-party service, authentication can use:

- API keys
- OAuth 2.0
- JWT
- Client certificates
- Signed requests

Sensitive credentials should be stored securely and should not be exposed in client-side code.

Example Payment Request

```
POST /payment
Content-Type: application/json
Authorization: Bearer ACCESS_TOKEN
{
  "amount": 1500,
  "currency": "INR",
  "orderId": "ORD1001"
}
```

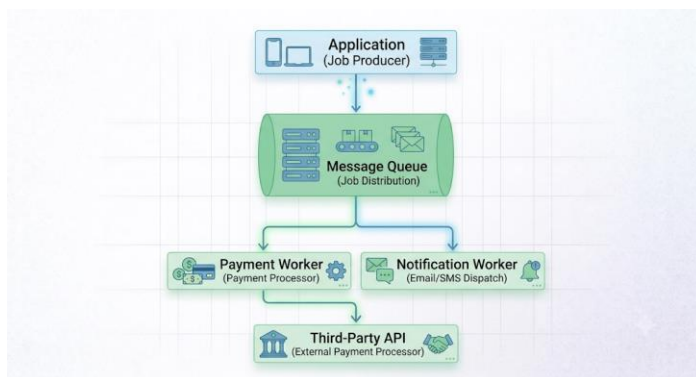
Response Processing

```
{
  "transactionId": "TXN501",
  "status": "SUCCESS"
}
```

The application verifies the response and updates the transaction status.

Alternative Approach: Asynchronous Integration

For operations that do not require an immediate response, a **message queue** can be used.



Benefits

- Reduces waiting time.
- Handles temporary third-party failures.
- Supports retries.
- Prevents sudden traffic spikes from overwhelming external services.

- Improves overall reliability.

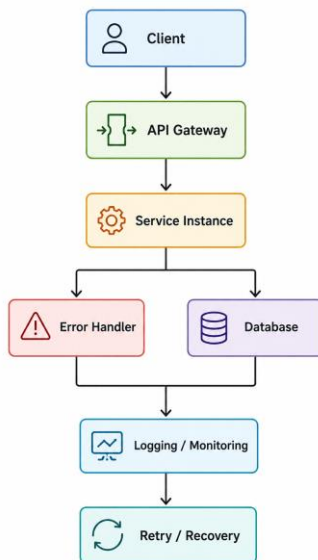
Conclusion

Using an **API Gateway** + **secure authentication** + **integration service** provides controlled third-party access. For high-volume or non-immediate operations, asynchronous queues and retry mechanisms can improve efficiency and reliability.

5. Error Handling and Fault Tolerance

A web service that frequently encounters errors requires systematic error handling and fault-tolerance mechanisms.

Proposed Architecture



Types of Errors

1. **Client errors** – Invalid input, authentication failure, invalid request.
2. **Server errors** – Internal application failure.
3. **Network errors** – Connection failure or timeout.
4. **Database errors** – Database unavailable or query failure.

5. **Third-party errors** – External API unavailable or returning an error.

Error Handling Mechanism

The service should return appropriate HTTP status codes.

Status	Meaning
200	Successful request
400	Bad request
401	Unauthorized
403	Forbidden
404	Resource not found
408	Request timeout
429	Too many requests
500	Internal server error
502	Bad gateway
503	Service unavailable

Standard Error Response

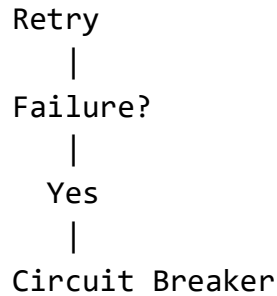
```
{
  "error": {
    "code": "USER_NOT_FOUND",
    "message": "The requested user does not exist.",
    "requestId": "REQ10025"
  }
}
```

Fault Tolerance Techniques

1. *Retry Mechanism*

Temporary failures can be retried using controlled retry attempts.

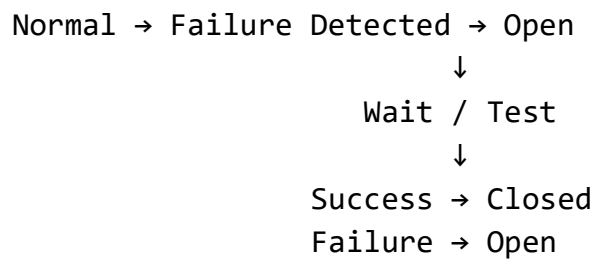
```
Request
|
Failure?
|
Yes
|
```



Exponential backoff should be used to prevent excessive repeated requests.

2. Circuit Breaker

If a dependent service repeatedly fails, the circuit breaker temporarily stops sending requests to it.



3. Timeout

Every external service request should have a maximum waiting time.

4. Load Balancing

Requests can be distributed across multiple service instances so that failure of one server does not stop the complete application.

5. Logging and Monitoring

Errors should be recorded with:

- Timestamp
- Request ID
- Error code
- Service name
- Status code

- Relevant diagnostic information

Sensitive information such as passwords and authentication tokens should never be written to logs.

6. Graceful Degradation

If a non-critical service fails, the main application should continue operating.

For example, if a recommendation service fails, the shopping application can still allow users to browse and purchase products.

Alternative Reliability Strategy

A highly reliable system can combine:

Load Balancer
+
Multiple Servers
+
Caching
+
Retry + Exponential Backoff
+
Circuit Breaker
+
Message Queue
+
Monitoring

Conclusion

A combination of **standardized error responses, timeouts, retries, circuit breakers, load balancing, logging, monitoring, and graceful degradation** can significantly improve web-service reliability and provide a smoother user experience.

Overall Conclusion

For distributed web services, the appropriate technology should be selected according to the application requirements:

- **REST** → lightweight mobile/web APIs.
- **SOAP** → structured enterprise services requiring standards such as WS-Security.
- **HTTPS/TLS** → secure client-server communication.
- **API Gateway and Microservices** → scalability and service integration.
- **Retries, circuit breakers, queues, and monitoring** → fault tolerance and reliability.
- **WSDL and XML Schema** → formal service contracts and structured interoperability when SOAP-based services are used.